

Information Processing 1 — Week 11 exercises

Complete each exercise on your own laptop before the end of the exercise class. When you finish an exercise, **immediately** ask an instructor to check your answer. Answers checked after the end of the class will have a 50% penalty.

Do not struggle for more than a few minutes with anything you cannot solve. If you are stuck or have any questions then **raise your hand** and an instructor will help you.

1 [4 points] Using dictionaries to count things

In class we wrote a program to count the frequency of each letter in a string:

```
s = "peter piper picked a peck of pickled peppers"
d = dict()
for c in s:
    if not c in d:
        d[c] = 0
    d[c] = d[c] + 1
print(d)
```

1.1 Record a histogram while analysing data

Generalise this program by converting the loop into a function `histogram(seq)` that can create a histogram of the elements in any collection. Test your function by storing the histogram for the string used above in a variable called `h` and then printing it.

1.2 Print the content of a histogram in order

Write a function `printAscending(d)` that prints the keys and values of `d` in order, using the sorted keys of `d`. Test your function by printing `h` in ascending order of its keys. Make sure the output lists keys in *alphabetical* order. For example, if `d` is { "b":1, "c":2, "a":3 } then `printAscending(d)` will produce:

```
a 3
b 1
c 2
```

Test your function by using it to print the variable `h` from the previous question.

1.3 Display a histogram as a bar chart

The values stored in the histogram `h` are all numbers. Write a function called `printHistogram(d)` that prints each key in `d` (in ascending order) followed by a line of asterisks '*' where the number of '*'s is controlled by the value associated with the key in the dictionary.

For example, if `d` is { "b":1, "c":2, "a":3 } then `printHistogram(d)` will produce:

```
a ***  
b *  
c **
```

Test your function by using it to print the variable `h` (from the previous question).

1.4 Using `dict.get()` to create keys when they are missing

The `histogram(seq)` function contains an `if` statement that checks if a key is not already in the dictionary and, if necessary, creates an entry for it with value 0. This ensures an entry always exists for the next part of the program that adds one to the appropriate entry. This is the same as looking up the key using `dict.get(key, default)` with a `default` value of 0, then adding one to the result and storing it back in the dictionary at the same key.

Improve your `histogram(seq)` function by replacing the three lines in the body of the `for` loop with a single line that uses `dict.get()` to calculate the value to store back into the dictionary at the appropriate key.

▷▷ Ask an instructor to check and record your work **now** (before you run out of time).

2 [2 points] Mapping data from one domain to another

A dictionary can be used as a ‘discrete function’ where the input parameter is the dictionary key and the result is the value stored at that key. You can use such as dictionary as a map from one domain to different codomain. A traditional printed dictionary, mapping one language onto another, is exactly this kind of map.

The inverse function, mapping the codomain back to the original domain, is also useful and can be constructed automatically from the original map.

2.1 Create a map and a reverse map for English and Japanese

For this section you will need to create a dictionary called `e2j` that maps the English words for the first five numbers onto the corresponding Japanese words. Create that dictionary now.

2.2 Use the map to create a reverse map for Japanese to English

In the map of number words `e2j` there is an entry `"two": "ni"`. In the reverse map there will have to be a corresponding entry `"ni": "two"`. The same requirement holds for all other entries in the original map.

Write a function called `reverseMap(d)` that returns a new dictionary whose *keys* are the *values* of `d`, and whose corresponding *values* are the *keys* of `d`.

Test your function by printing out both the ‘forward’ (EN to JA) and the ‘reverse’ (JA to EN) maps of the number words.

2.3 Creating the reverse map of a non-invertible function

The `reverseMap()` function in the previous exercise works fine when the original map has unique keys and unique values, as in `e2j`.¹ The `reverseMap()` function does not work well to reverse the histogram of letter frequencies, `h` in question 1.1, because more than one letter can have the same frequency.²

Verify the problem by printing the histogram `h` and its reverse map, which should be missing some entries.

```
print(h)
print(reverseMap(h))
```

For example, 'i' and 'c' and 'k' all appear three times in the string. In the reverse map however there is only one letter associated with the entry for 3.

Write a new `reverseMap(d)` function that uses the values of `d` as the keys in the reverse map, as before. Unlike before, for each key in the reverse map use a *list* as its value in the map. The list will contain all possible values for that key. Ambiguity is therefore permitted because each list will contain all the keys in `d` that map to that key in the reverse map.

For example, in the reverse map of `h` there will be an entry for 3: ['i', 'c', 'k'] indicating that the three keys 'i', 'c', and 'k' all have the value 3 in the original histogram `h`. Use `printAscending()` to print the `reverseMap()` of `h` in a nice order.

▷▷ Ask an instructor to check and record your work **now** (before you run out of time).

3 [2 points] Optimisation and performance measurement

The following code measures how long it takes (in seconds) to calculate the 32nd Fibonacci number. (It should take about one second. If it takes longer, reduce the argument; if it is too fast, increase the argument.)

```
def fib(n):
    if n < 2:
        return 1
    return fib(n-1) + fib(n - 2)

import time

start = time.time()
print(fib(32)) # adjust the argument as needed to take 1 second
print(time.time() - start)
```

The `fib(n)` function is *pure*. This means that for the *same argument* value it will *always* return the *same result*. We can use this characteristic to speed it up, by 'memoising' results in a dictionary whose keys are the arguments `n` given to `fib(n)` and whose values are the corresponding results.

¹This works because the original mapping is a *bijection* meaning it is a 'one-to-one' mapping from domain to co-domain with the same number of elements in each. Each key in the domain maps to a single, unique value in the co-domain and each value in the co-domain has a single, unique corresponding key in the domain. Reversing the mapping is therefore trivial. Any function f must be a bijection for its inverse function f^{-1} to exist.

²The histogram is not a bijection, it is a 'many-to-one' mapping meaning that multiple elements in the domain (dictionary keys) can map to the same element (dictionary value) in the co-domain. When you attempt to reverse the mapping you find values in the co-domain that map to multiple keys in the domain, meaning the inverse function is ambiguous for that value. The inverse function therefore does not exist, because ambiguity is not permitted for any function.

3.1 Memoising previous results of 'pure' functions

Add a `memo` dictionary to the `fib(n)` function. The dictionary should be a global variable, defined before and outside the `fib(n)` function. At the start of `fib(n)`, check if `memo` has an entry for the key `n`. The first time `fib(n)` is called with a particular argument `n`, `n` will not be a key in `memo`. The result should be calculated normally, added to the `memo` dictionary at key `n`, and then returned. Subsequent calls of `fib(n)` with the same argument `n` will retrieve the previously calculated result from `memo` and return it immediately, instead of calculating it again.

Once your memo is working, and the function is producing the correct results, time it using the same code as above.

How many times faster is your memoised version, compared to the original non-memoised version?

▷▷ Ask an instructor to check and record your work now (before you run out of time).

4 [2 points] Analysing and displaying information about a text file

For this section you will need to download the file `text2.txt` to a folder where Python can find and `open()` it.

A dictionary can be used to count the frequency of words in a file (similar to the way we counted letters in a string earlier). For each line of text in the file, first remove anything that is not a letter or a space. Then split the line into a list of words. Use each word, converted to lower case, as a key in a dictionary whose values count the number of times that each word has been seen in the file.

This is similar to the histogram of letters in a string with the following differences:

1. there are multiple lines read from a file (instead of just one string),
2. each line must be 'depunctuated' and then split into a list of individual words,
3. the list of words is enumerated (instead of individual characters in the single string), and
4. the lowercase version of each word is used as the key in the dictionary (instead of using a single character).

The next three exercises lead you through the development of this program.

4.1 Differentiate letters and non-letters

The method `c.isalpha()` returns `True` if `c` is a letter of the alphabet. The method `c.isspace()` returns `True` if `c` is a space.

Write a function `isLetter(c)` that returns `True` if `c` is either a letter or a space. Test your function on several examples:

```
print(isLetter('a')) # True
print(isLetter('B')) # True
print(isLetter(' ')) # True
print(isLetter(',')) # False
print(isLetter('.')) # False
```

4.2 Write a function to remove punctuation from text

The function `filter(function, string)` calls `function(c)` for each character `c` in the `string` and then returns a list of only those characters for which `function(c)` returned `True`.

The method `sep.join(l)` creates a string made from all of the characters in the list `l` separated by the string `sep`. The string `sep` can be empty, in which case `''.join(l)` creates a string from the list of characters in `l`.

Write a function `depunctuate(word)` that removes everything from `word` that is not a letter or a space.

You can use `filter()` and your function `isLetter()` to do this. Test your program on each line in the file, printing out the de-punctuated version, and making sure there are only words and spaces left in the depunctuated lines.

4.3 Creating a histogram of words in text

Make a histogram of the word frequencies in the text file `text2.txt`. To your previous answer you will have to add a dictionary to hold the histogram, a call to `line.split()` to separate the de-punctuated `line` into a list of words, and a loop that iterates over the list of words and increments a corresponding frequency count in the dictionary. (The loop will look very similar to the one in your letter counting program.) Use `printAscending()` to print out the resulting histogram and verify that the words “do” and “know” both appear seven times in the file.

▷▷ Ask an instructor to check and record your work now (before you run out of time).

5 Challenge [1 bonus point]

Write a program that reports the number of lines, words, and characters in the file `text2.txt`. Consider a word to be any sequence of non-space characters, i.e., those for which `c.isspace()` returns `False`. (Note that this is a different interpretation of word compared to the previous questions. It is the same interpretation of ‘word’ that `str.split()` uses. The total count of characters should count all characters, including spaces and newlines.)

Hint: If you write the program as economically as possible, you should not need to define any functions of your own and the body of the main loop should contain only three lines of code.

Verify that `text2.txt` contains 41 lines, 422 words, and 2480 characters.

▷▷ Ask an instructor to check and record your work now (before you run out of time).